

```
In [103]: # Initialize Otter
import otter
grader = otter.Notebook("proj1.ipynb")
```

1 Project 1 - SQL

1.1 Due Date: Thursday, September 21st, 5:00pm

In this project, we will be working with SQL on the IMDB database.

1.2 Objectives

- Explore and extract relevant information from database with SQL functions
- Perform data cleaning and transformation using string functions and regex
- Use the cleaned data to run insightful analysis using joins, aggregations, and window functions

Note: If at any point during the project, the internal state of the database or its tables have been modified in an undesirable way (i.e. a modification not resulting from the instructions of a question), restart your kernel, clear output, and simply re-run the notebook as normal. This will shutdown your current connection to the database, which will prevent the issue of multiple connections to the database at any given point. When re-running the notebook, you will create a fresh database based on the provided Postgres dump.

1.3 Logistics & Scoring Breakdown

Each coding question has **both public tests and hidden tests**. Roughly 50% of your grade will be made up of your score on the public tests released to you, while the remaining 50% will be made up of unreleased hidden tests. In addition, there are two free-response questions that will be manually graded.

This is an **individual project**. However, you're welcome to collaborate with any other student in the class as long as it's within the [academic honesty guidelines](#). Create new cells as needed to acknowledge others.

Question	Points
0	1
1a	1
1b	2
1c	1
1d	1
2a	1
2b	3

Question	Points
2c	3
3a	2
3b	2
3c	2
3d	1
4a	2
4b	2
4c	1
5	2
Total	27

```
In [104]: # Run this cell to set up imports
import numpy as np
import pandas as pd
```

2 Before You Start: Assignment Tips

Please Read!! In this project we will assume you have attended lecture and seen how to connect to a Postgres server via two ways: JupySQL in Jupyter Notebook, and the psql command-line program.

We have written up these instructions for you in the Fall 2023 Assignment Tips—a handy resource that has many other tips:

- PostgreSQL documentation
- JupySQL and magic commands in Jupyter
- JupyterHub keyboard shortcuts
- psql and common meta-commands
- Debugging:
 - Where to create new cells to play nice with the autograder
 - Opening/closing connections, deleting databases if all else fails
- Local installation (not supported by staff officially, but for your reference)

For some questions with multi-line cell magic, we will also be saving the literal query string with `query snippets` using `--save`:

```
%sql --save query result << select * FROM table ...
```

3 Database Setup

We are going to be using the JupyterSQL library to connect our notebook to a PostgreSQL database server on your JupyterHub account. Running the next cell will do so; you should not see any error messages after it executes.

```
In [105]: # The first time you are running this cell, you may need to run the following line as: %load_ext
          %reload_ext sql
```

In the next cell, we will unzip the data. This only needs to be done once.

```
In [106]: !unzip -u data/imdbdb.zip -d data/
```

Archive: data/imdbdb.zip

Create the imdb database: We will use PostgreSQL commands to create a database and import our data into it. Run the following cell to do this. * You can also run these cells in the command-line via `psql`. * If you run into the **role does not exist** error, feel free to ignore it. It does not affect data import.

```
In [107]: !psql postgresql://jovyan@127.0.0.1:5432/imdb -c 'SELECT pg_terminate_backend(pg_stat_activity->pid)
          !psql postgresql://jovyan@127.0.0.1:5432/postgres -c 'DROP DATABASE IF EXISTS imdb'
          !psql postgresql://jovyan@127.0.0.1:5432/postgres -c 'CREATE DATABASE imdb'
          !psql postgresql://jovyan@127.0.0.1:5432/imdb -f data/imdbdb.sql
```

```
pg_terminate_backend
```

```
-----
```

```
t
(1 row)
```

```
DROP DATABASE
CREATE DATABASE
SET
SET
SET
SET
SET
```

```
set_config
```

```
-----
```

```
(1 row)
```

```
SET
```

```
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
COPY 500000
COPY 3804162
COPY 113
COPY 2433431
COPY 337179
COPY 12
ALTER TABLE
ALTER TABLE
```

Connect to imdb database in the Notebook: Now let's connect to the new database we just created! There should be no errors after running the following cell.

```
In [108]: %sql postgresql://jovyan@127.0.0.1:5432/imdb
```

Connect to imdb database in psql:

Do the following in a Terminal window!

Connect to the same database via `psql`. See the [Fall 2023 Assignment Tips](#) website resource for details on connecting. Run the following meta-command in the `psql` client:

```
\l
```

This should display all databases on this server, including the `imdb` database you just created.

Quick check: To make sure things are working, let's fetch 10 rows from one of our tables `cast_sample`.

Just run the following cell, no further action is needed.

```
In [109]: %%sql
          SELECT *
          FROM cast_sample
          LIMIT 10
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

```
/srv/conda/envs/notebook/lib/python3.11/site-packages/sql/connection/connection.py:827: JupySQLRollbackWarning:
warnings.warn(
```

10 rows affected.

```
Out[109]: +-----+-----+-----+-----+
| id | person_id | movie_id | role_id |
+-----+-----+-----+-----+
| 708 | 235 | 2345369 | 1 |
| 721 | 241 | 2504309 | 1 |
| 789 | 264 | 2156734 | 1 |
| 875 | 299 | 1954994 | 1 |
| 888 | 302 | 765037 | 1 |
| 889 | 302 | 765172 | 1 |
| 898 | 306 | 291387 | 1 |
| 899 | 306 | 1477434 | 1 |
| 931 | 324 | 824119 | 1 |
| 1936 | 543 | 1754068 | 1 |
+-----+-----+-----+-----+
Truncated to displaylimit of 10.
```

3.1 Connect to the grader

```
In [110]: # Connecting the grader
          # Just run the following cell, no further action is needed.
          from data101_utils import GradingUtil
          grading_util = GradingUtil("proj1")
          grading_util.prepare_autograder()
```

3.2 The imdb Database

In this project, we are working with a reduced version of the Internet Movie Database (IMDb) database. This Postgres database is a small random sample of actors from the much larger full database (which is over several GBs large) and includes their corresponding movies and cast info. Disclaimer: as a result, we may obtain wildly different results than if we were to use the entire database.

- **actor_sample**: information about the actors including id, name, and gender
- **cast_sample**: each person on the cast of each movie gets a row including cast id, each person's id (`actor_sample.id`), movie id (`movie_sample.id`), and role id
- **movie_sample**: sample of movies the actors have been in, including movie id, title, and the production year
- **movie_info_sample**: this table originally had a lot of information for each movie (take a look at `info_type` to see the information available), but we have dropped some information to make it easier to manage. This table includes movie info's id, movie id, info type id, and the info itself
- **info_type**: reference table to match each info type id to the description of the type of information
- **role_type**: reference table for `cast_sample` to match role id to the description of the role

3.2.1 Key Notes

- This database is **not** the same as the IMDb lecture database, but has a lot of similar features.
- Point of confusion: `movie_sample` and `actor_sample` both have attributes `id` corresponding to 7 digit unique numeric identifiers, but do **not** refer to the same data values.
- `cast_sample` is analogous to the `crew` table from lecture. It can be used to match an actor's id to movies they have acted in, among other relations.
- You are highly encouraged to spend some time exploring the metadata of these tables using Postgres meta-commands to better understand the data given and the relations between tables.

4 The information_schema schema

A **schema** is a namespace of tables in the database, often used for security purposes. Let's see how many schema are defined for us in our current database:

```
In [111]: %%sql
          SELECT *
          FROM information_schema.schemata;
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

4 rows affected.

```
Out[111]: +-----+-----+-----+-----+-----+
| catalog_name | schema_name | schema_owner | default_character_set_catalog | default_
+-----+-----+-----+-----+-----+
| imdb         | pg_toast    | jovyan       | None                          |
| imdb         | pg_catalog  | jovyan       | None                          |
| imdb         | public      | jovyan       | None                          |
| imdb         | information_schema | jovyan       | None                          |
+-----+-----+-----+-----+-----+
```

Within a Postgres database, there are often at least three schemas: * **public**, a public schema that users can access and create tables in; * **pg_catalog**, a schema for maintaining system information; and * **information_schema**, a schema that maintains metadata about objects currently created in the database. * The fourth schema **pg_toast** maintains data that can't regularly be stored in relations, such as very large data values. See more in documentation [here](#).

For now, we focus on the **information_schema** schemata, which stores our metadata. That's right—metadata is also data, and as we make updates to our **public** databases, metadata is automatically stored and updated into different tables under the **information_schema** schema.

There are many metadata tables that Postgres updates for us, and the full list is in the Postgres documentation ([Chapter 37](#)). For now, let's look at which the **.tables** table ([37.54](#)), which lists all the tables located in the database. Let's specifically look at those that are in the public schema (i.e., publicly accessible tables):

```
In [112]: %%sql
SELECT *
FROM information_schema.tables
WHERE table_schema = 'public';
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

6 rows affected.

```
Out[112]: +-----+-----+-----+-----+-----+
| table_catalog | table_schema | table_name | table_type | self_referencing_column_name
+-----+-----+-----+-----+-----+
| imdb         | public      | actor_sample | BASE TABLE | None
| imdb         | public      | cast_sample  | BASE TABLE | None
| imdb         | public      | info_type   | BASE TABLE | None
| imdb         | public      | movie_info_sample | BASE TABLE | None
| imdb         | public      | movie_sample | BASE TABLE | None
| imdb         | public      | role_type   | BASE TABLE | None
+-----+-----+-----+-----+-----+
```

5 Question 0

As stated above, there are many metadata tables stored in the `information_schema` schema. Write a query that returns the names of all relations in the PostgreSQL `information_schema` schema, i.e., the names of all the metadata tables

Hints: * Your resulting table names should correspond to what's listed in the information schema documentation ([Chapter 37](#)). * For you to think about: Why might there be fewer tables in your query response than the full list in the documentation?

```
In [113]: %%sql --save query_0 result_0 <<
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'information_schema'
```

Running query in 'postgres://jovyan@127.0.0.1:5432/imdb'

69 rows affected.

```
In [114]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_0 = %sqlcmd snippets query_0
grading_util.save_results("result_0", query_0, result_0)
result_0
```

```
Out[114]: +-----+
|          table_name          |
+-----+
| information_schema_catalog_name |
|          attributes          |
|        applicable_roles        |
| administrable_role_authorizations |
| check_constraint_routine_usage |
|        character_sets         |
|        check_constraints       |
|        collations              |
| collation_character_set_applicability |
|        column_column_usage    |
+-----+
```

```
In [115]: grader.check("q0")
```

Out[115]: q0 results: All test cases passed!

6 Question 1: Exploratory Data Analysis

One of the first things you'll want to do with a database table is get a sense for its metadata: column names and types, and number of rows.

6.1 Tutorial

We can use the PostgreSQL `\d` meta-command to get a description of all the columns in the `movie_info_sample` table. Open up a terminal window, connect to the imdb server, and analyze the output of the meta-command:

```
\d movie_info_sample
```

We can use the PostgreSQL `\d` meta-command to get a description the `movie_info_sample` schema. Open up a terminal window, connect to the imdb server, and analyze the output of the meta-command:

```
\d movie_info_sample
```

There are four attributes in this schema, of which "id" is one. What are the other attribute names? Assign `result_1a` to a list of strings, where each element is an attribute name. The list does not need to be in order.

```
In [116]: result_1a = ["id", "movie_id", "info_type_id", "info"]
          result_1a
```

```
Out[116]: ['id', 'movie_id', 'info_type_id', 'info']
```

```
In [117]: grader.check("q1a")
```

```
Out[117]: q1a results: All test cases passed!
```

Debugging tip: Throughout this project and when working with databases, you should always be checking schemas via the `\d psql` metacommand.

6.2 Question 1b

Next, let's continue with our initial exploration of this table. How many rows are in this table?

Assign `result_1b` to the result of a SQL query to calculate the number of rows in the `movie_info_sample` table. Then, assign `count_1b` to the integer number of rows based on what you found in `result_1b`. Do not hard code this value.

Hints: - See the [Assignment Tips](#) page for how to use SQL line magic. - Your query result should have exactly one row and one attribute; the lone value in the instance should be the number of rows. - See the [JupyterSQL documentation](#) for how to index into a SQL query result.

```
In [118]: result_1b = %sql SELECT COUNT(*) AS number_of_rows FROM movie_info_sample;
          count_1b = result_1b[0][0]

          # do not edit below this line
          display(result_1b)
          count_1b
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

1 rows affected.

```
+-----+
| number_of_rows |
+-----+
|      2433431   |
+-----+
```

Out[118]: 2433431

```
In [119]: grader.check("q1b")
```

Out[119]: q1b results: All test cases passed!

6.3 Question 1c: Random table sample

Now that we know a bit about the metadata of the table, let's randomly sample rows from `movie_info_sample` to explore its contents.

Given that you know the size of the table from the previous query, **write a query that retrieves 5 tuples on expectation using the BERNOLLI sampling method.** That is, if we run the query multiple times, we should get 5 tuples on average in our resulting table. The BERNOLLI sampling method scans the whole table and selects individual rows independently with `p%` probability. Please see the [documentation](#) for syntax.

Hints/Details: * Assign `p_1c` to a sampling rate that you pass into the `query_1c` f-string using Python variable substitution. Your formula should contain `count_1b`. Don't forget to express `p_1c` in units of percent, i.e., `p_1c = 0.03` is 0.03%! * For a refresher on f-strings and Python variable substitution, see [this tutorial](#). If Python variable substitution is done correctly, we should be able to change our `p%` probability by simply reassigning `p_1c` and rerunning the query. (Please leave `p_1c` unchanged.) * We have completed the SQL line magic for you; this references the Python f-string `query_1c` you created within a SQL query using JupyterSQL-specific syntax. * Try running the SQL cell many times and see what you notice.

```
In [120]: p_1c = (5 / count_1b) * 100
          query_1c = f"SELECT * FROM movie_info_sample TABLESAMPLE BERNOULLI ({p_1c});" # edit this query

          # Do not edit below this line
          result_1c = %sql {{query_1c}}
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

1 rows affected.

```
In [121]: # Do not delete/edit this cell!
          # You must run this cell before running the autograder.
          grading_util.save_results("result_1c", query_1c, result_1c)
          result_1c
```

```
Out[121]: +-----+-----+-----+-----+
          |    id    | movie_id | info_type_id | info |
          +-----+-----+-----+-----+
          | 6200595 | 2011523 |          3   | Drama |
          +-----+-----+-----+-----+
```

```
In [122]: grader.check("q1c")
```

```
Out[122]: q1c results: All test cases passed!
```

6.4 Question 1d: Random sample, fixed number of rows

If a random number of rows is not of importance, a more efficient way to get some arbitrary tuples from a table is to use the `ORDER BY` and `LIMIT` clauses. In the next cell, fetch 5 **random** tuples from `movie_info_sample`. Compared to the previous question, your query result here should always have 5 tuples!

Hint: Check out lecture.

```
In [123]: %%sql --save query_1d result_1d <<
SELECT *
FROM movie_info_sample
ORDER BY RANDOM()
LIMIT 5
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

5 rows affected.

```
In [124]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_1d = %sqlcmd snippets query_1d
grading_util.save_results("result_1d", query_1d, result_1d)
result_1d
```

```
Out[124]: +-----+-----+-----+-----+
|    id    | movie_id | info_type_id | info |
+-----+-----+-----+-----+
| 6851968 | 2419280 | 3 | Short |
| 9315280 | 1812080 | 1 | 16 |
| 1554434 | 2288275 | 106 | $443,003 (USA) (18 November 2012) (16 screens) |
| 4178388 | 72932 | 8 | USA |
| 9361742 | 1880139 | 1 | Poland:94 |
+-----+-----+-----+-----+
```

```
In [125]: grader.check("q1d")
```

```
Out[125]: q1d results: All test cases passed!
```

7 Question 2: Data Cleaning

The `movie_sample` table contains a very minimal amount of information per movie:

```
In [126]: %sql SELECT * FROM movie_sample LIMIT 5;
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

5 rows affected.

```
Out[126]: +-----+-----+-----+
|    id    |      title      | production_year |
+-----+-----+-----+
| 2038405 | La corte de faraón |      1944      |
| 2081186 | Long de xin      |      1985      |
| 2177749 | Onésime aime les bêtes |      1913      |
| 1718608 | Bedtime Worries  |      1933      |
| 2130699 | Mothman           |      2000      |
+-----+-----+-----+
```

In this question, we're going to create a nice, refined view of the `movie_sample` table that also includes a **rating field**, called `movie_ratings`.

The [MPAA rating](#) is commonly included in most datasets about movies, including ours, but in its current format in the dataset, it's quite difficult to extract.

The first clue about our approach comes from the random rows you explored in Question 1. As you saw, the `movie_info_sample` table contains a lot of information about each movie. Each row contains a particular type of information (e.g., runtime, languages) categorized by `info_type_id`. Based on the other tables in this database, the `info_type` table is a reference table to this ID number.

Our strategy in this question is therefore as follows: * **Question 2a:** Find the `mpaa_rating_id` from the `info_type` table. * **Question 2b:** Extract the MPAA rating of a specific movie from the `movie_info_sample` table. * **Question 2c:** Construct a view `movie_ratings` based on the `movie_sample` table and all relevant MPAA ratings extracted from the `movie_info_sample` table.

7.1 Question 2a: MPAA Rating and info_type

To start, using the `info_type` table, write a query to find which `id` corresponds to a film's MPAA rating. The query `result_2a` that you write should return a relation with exactly one row and one attribute; the lone value in the instance should be the MPAA rating id number. We've then assigned `mpaa_rating_id` to extract the number itself from the relation.

Hints: - Open the `psql` client in a terminal to explore the schema of `info_type` via the `\d` metacommand (see the [Assignment Tips](#) page). Remember you can also write SQL commands to that terminal to interact with the IMDB database, but all final work must be submitted through this Jupyter Notebook. - Be careful when using quotes. SQL interprets single and double quotes differently. The single quote character `'` is reserved for delimiting string constants, while the double quote `"` is used for naming tables or columns that require special characters. See [documentation](#) for more.

```
In [127]: result_2a = %sql SELECT id FROM info_type WHERE info = 'mpaa';
          mpaa_rating_id = result_2a[0][0]

          # do not edit below this line
          display(result_2a)
          mpaa_rating_id
```

Running query in 'postgres://jovyan@127.0.0.1:5432/imdb'

1 rows affected.

```
+-----+
| id |
+-----+
| 97 |
+-----+
```

Out[127]: 97

```
In [128]: grader.check("q2a")
```

Out[128]: q2a results: All test cases passed!

7.2 Question 2b: Looking up the MPAA Rating

Suppose we wanted to find the MPAA rating for the 2004 American teen drama classic, *Mean Girls*. The below cell assigns `movie_id_2b` to the IMDb ID of this movie, 2109683.

```
In [129]: # Just run this cell, no further action is needed.
movie_id_2b = 2109683
%sql SELECT * FROM movie_sample WHERE id = {{movie_id_2b}};
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

1 rows affected.

```
Out[129]: +-----+-----+-----+
|   id   |  title   | production_year |
+-----+-----+-----+
| 2109683 | Mean Girls |         2004    |
+-----+-----+-----+
```

In the next cell, **write a query to find the MPAA rating for this movie**. Your query should return a relation with exactly one row, which has (`info`, `mpaa_rating`), where `info` is the full MPAA rating string from `movie_info_sample`, and `mpaa_rating` is just the rating itself (i.e. R, PG-13, PG, etc) for this movie.

Before you get started: * Explore the `movie_info_sample` tuples corresponding to the MPAA rating by using metacommands in the terminal. The `info` field is a little longer than just the rating. It also includes an explanation for why that movie received its rating. * You will need to extract a substring from the `info` column of `movie_info_sample`; you can use the [string functions](#) in PostgreSQL to do it. There are many possible solutions. One possible solution is to use the substring function along with regex. If you use this approach, [this section on regex](#) may be particularly useful. [regex101.com](#) may also be helpful to craft your regular expressions. * You may use `mpaa_rating_id` and `movie_id_2b` directly in the rest of the questions using Python variable substitution (i.e., double curly braces). See the JupyterSQL documentation for more details.

```
In [130]: %%sql --save query_2b result_2b <<
SELECT info,
       --substring(string from pattern for escape-character)
       SUBSTRING(info FROM ' (.* for') AS mpaa_rating
FROM movie_info_sample
WHERE movie_id = {{movie_id_2b}} AND info_type_id = {{mpaa_rating_id}};
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

1 rows affected.

```
In [131]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_2b = %sqlcmd snippets query_2b
grading_util.save_results("result_2b", query_2b, result_2b)
result_2b
```

```
Out[131]: +-----+-----+
| info | mpaa_rating |
+-----+-----+
| Rated PG-13 for sexual content, language and some teen partying | PG-13 |
+-----+-----+
```

```
In [132]: grader.check("q2b")
```

```
Out[132]: q2b results: All test cases passed!
```

You may use `mpaa_rating_id` directly in the rest of the questions using python variable substitution.

7.3 Question 2c

In the next cell, 1. Construct a view named `movie_ratings` containing one row for each movie, which has (`movie_id`, `title`, `info`, `mpaa_rating`), where `info` is the full MPAA rating string from `movie_info_sample`, and `mpaa_rating` is just the rating itself (i.e. R, PG-13, PG, etc). * In other words, extend `movie_sample` with the MPAA rating attributes that you found in the previous question part, but this time for all movies. 2. Following the view definition, also write a `SELECT` query to return the **first 20 rows** of the view, ordered by ascending `movie_id`.

```
In [133]: %%sql
SELECT info_type_id
FROM movie_info_sample;
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

2433431 rows affected.


```
Out[133]: +-----+
| info_type_id |
+-----+
|      1      |
|      1      |
|      1      |
|      1      |
|      1      |
|      1      |
|      1      |
|      1      |
|      1      |
|      1      |
+-----+
Truncated to displaylimit of 10.
```

```
In [134]: %%sql
SELECT *
FROM movie_sample
JOIN movie_info_sample ON movie_sample.id = movie_info_sample.movie_id
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

1573145 rows affected.

```
Out[134]: +-----+-----+-----+-----+-----+-----+-----+
| id | title | production_year | id_1 | movie_id | info_type_id | info |
+-----+-----+-----+-----+-----+-----+-----+
| 1632850 | #1 | 2005 | 9200121 | 1632850 | 1 | USA:30 |
| 1632850 | #1 | 2005 | 5610379 | 1632850 | 3 | Drama |
| 1632850 | #1 | 2005 | 4542002 | 1632850 | 8 | USA |
| 1632850 | #1 | 2005 | 1486002 | 1632850 | 105 | $5,000 |
| 1632850 | #1 | 2005 | 5610380 | 1632850 | 3 | Short |
| 1632851 | #1 | 2010 | 5610381 | 1632851 | 3 | Comedy |
| 1632851 | #1 | 2010 | 4542003 | 1632851 | 8 | USA |
| 1632851 | #1 | 2010 | 5610382 | 1632851 | 3 | Short |
| 1632851 | #1 | 2010 | 1681925 | 1632851 | 105 | $250 |
| 1632851 | #1 | 2010 | 9200122 | 1632851 | 1 | USA:12 |
+-----+-----+-----+-----+-----+-----+-----+
Truncated to displaylimit of 10.
```

```
In [135]: %%sql --save query_2c result_2c <<
DROP VIEW IF EXISTS movie_ratings;
CREATE VIEW movie_ratings AS (
SELECT
    movie_id,
    title,
```

```

        info,
        SUBSTRING(info FROM 'Rated (.* for') AS mpaa_rating
FROM movie_sample
JOIN movie_info_sample ON movie_sample.id = movie_info_sample.movie_id
WHERE movie_info_sample.info_type_id = {{mpaa_rating_id}}
);

SELECT * FROM movie_ratings
ORDER BY movie_id ASC
LIMIT 20;

```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

20 rows affected.

```

In [136]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_2c = %sqlcmd snippets query_2c
grading_util.save_results("result_2c", query_2c, result_2c)
result_2c

```

```

Out[136]: +-----+-----+-----+-----+
| movie_id |      title      |                                     info                                     |
+-----+-----+-----+-----+
| 1632926 | $5 a Day        | Rated PG-13 for sexual content and language |
| 1632941 | $9.99           | Rated R for language and behavior |
| 1632956 | $windle         | Rated R for some violence and language |
| 1633013 | 'A' gai wak     | Rated PG-13 for language |
| 1633014 | 'A' gai wak juk jap | Rated PG-13 for language |
| 1633461 | 'R Xmas         | Rated R for strong language, drug use, and sexual content |
| 1633618 | 'Til There Was You | Rated PG-13 for sensuality, language, and sexual content |
| 1633729 | (500) Days of Summer | Rated PG-13 for sexual content and language |
| 1633856 | (Untitled)      | Rated R for language |
| 1634282 | .45             | Rated R for pervasive strong language including graphic sexual content |
+-----+-----+-----+-----+

```

```

In [137]: grader.check("q2c")

```

```

Out[137]: q2c results: All test cases passed!

```

8 Question 3: Movie Moola

One measure of a movie's success is how much money it makes. If we look at our `info_type` table, we have information about the film's gross earnings and the budget for a film. It would be nice to know how much money a film made using the profit formula:

$$\text{profit} = \text{earnings} - \text{moneyspent}$$

We start by taking a look at the gross info type, with `info_type_id = 107`.

```
In [138]: %%sql
SELECT *
FROM movie_info_sample
WHERE info_type_id = 107
ORDER BY id
LIMIT 10 OFFSET 100000;
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

10 rows affected.

```
Out[138]: +-----+-----+-----+-----+
|      id      | movie_id | info_type_id | info                                     |
+-----+-----+-----+-----+
| 1464348 | 2281091 | 107 | INR 23,373,000 (India) (25 February 2005) |
| 1464349 | 2281091 | 107 | INR 19,207,000 (India) (18 February 2005) |
| 1464374 | 1766950 | 107 | HKD 826,364 (Hong Kong) (11 December 1975) |
| 1464375 | 1769023 | 107 | HKD 3,148,549 (Hong Kong) (19 November 1980) |
| 1464378 | 1799099 | 107 | HKD 6,493,694 (Hong Kong) (22 December 1981) |
| 1464383 | 1847670 | 107 | $21,438 (USA) (9 August 2009) |
| 1464384 | 1847670 | 107 | $10,266 (USA) (2 August 2009) |
| 1464396 | 1916002 | 107 | $5,932 (USA) (27 November 2005) |
| 1464397 | 1916002 | 107 | $4,206 (USA) (20 November 2005) |
| 1464398 | 1916002 | 107 | $2,939 (USA) (23 October 2005) |
+-----+-----+-----+-----+
Truncated to displaylimit of 10.
```

There are a lot of things to notice here. First of all, the values in the `info` attribute are strings with not only the earnings, but also the country and the month the earnings are cummulative summed until. Additionally, the info values are not all in the same currency! On top of that, it appears as if some of the gross earnings, even for those in USD are from worldwide sales, while others only count sales within the USA.

For consistency, let's only use movies with gross earnings counted in the USA and that are in US Dollars (\$).

In the next cell, - Construct a view named `movie_gross` containing one row for each movie, which has `(gross, movie_id, title)`, where `gross` is the numeric dollar amount extracted as a float. - To take a look at our cleaned data, write a `SELECT` query to display the **top 10 highest grossing films** from `movie_gross`.

```
In [139]: %%sql --save query_3a result_3a <<

DROP VIEW IF EXISTS movie_gross;
CREATE VIEW movie_gross AS

SELECT
    MAX(CAST(substring(regexp_replace(movie_info_sample.info, ',', '', 'g'), '\$(\d+) \$(US)', 'USD', '1000000000')) AS numeric) AS gross,
    movie_id,
    MAX(title) AS title

FROM movie_info_sample
JOIN movie_sample ON movie_info_sample.movie_id = movie_sample.id
WHERE info_type_id = 107 AND movie_info_sample.info LIKE '%$%'
GROUP BY movie_id

HAVING MAX(CAST(substring(regexp_replace(movie_info_sample.info, ',', '', 'g'), '\$(\d+) \$(US)', 'USD', '1000000000')) AS numeric) > 1000000000

SELECT *
FROM movie_gross
ORDER BY gross DESC
LIMIT 10;
```

10 rows affected.

20

```

query_3a = %sqlcmd snippets query_3a
grading_util.save_results("result_3a", query_3a, result_3a)
result_3a

```

```

Out[140]: +-----+-----+-----+
|   gross   | movie_id |           title           |
+-----+-----+-----+
| 760507625.0 | 1704289 |           Avatar          |
| 658672302.0 | 2438179 |           Titanic         |
| 623357910.0 | 2346436 |       The Avengers        |
| 534858444.0 | 2360583 |       The Dark Knight     |
| 460935665.0 | 2310522 |           Star Wars       |
| 448139099.0 | 2360588 | The Dark Knight Rises     |
| 436471036.0 | 2285018 |           Shrek 2         |
| 435110554.0 | 1851357 | E.T. the Extra-Terrestrial |
| 431065444.0 | 2310573 | Star Wars: Episode I - The Phantom Menace |
| 423315812.0 | 2204345 | Pirates of the Caribbean: Dead Man's Chest |
+-----+-----+-----+

```

```

In [141]: grader.check("q3a")

```

```

Out[141]: q3a results: All test cases passed!

```

8.2 Tutorial: Budget

We will now look at the budget info type, with `info_type_id = 105`.

```

In [142]: %%sql
SELECT *
FROM movie_info_sample
WHERE info_type_id = 105
ORDER BY id
LIMIT 10 OFFSET 5000;

```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

10 rows affected.

```
Out[142]: +-----+-----+-----+-----+
|      id      | movie_id | info_type_id |      info      |
+-----+-----+-----+-----+
| 1261074 | 1983149 | 105 | $75,000,000 |
| 1261110 | 1983269 | 105 | INR 180,000,000 |
| 1261160 | 2381188 | 105 | $40,000,000 |
| 1261170 | 1991083 | 105 | FIM 9,219,499 |
| 1261210 | 1993907 | 105 | $45,000,000 |
| 1261247 | 1995787 | 105 | $38,000,000 |
| 1261308 | 1999081 | 105 | $50,000,000 |
| 1261324 | 1999196 | 105 | SEK 40,000,000 |
| 1261375 | 2001114 | 105 | $60,000,000 |
| 1261396 | 2001989 | 105 | $13,000,000 |
+-----+-----+-----+-----+
Truncated to displaylimit of 10.
```

Similar to when we examined the gross info, we see a lot of non-US dollar currencies. For consistency, let's only use movies **with a budget in US dollars**.

8.3 Question 3b:

Now, we want something similar for the budget of the film, so that we can perform the subtraction of **gross** and **budget**. We want the numerical part of the **info** column and the **maximum budget value** for a particular film (as you can verify, some movies have more than one budget).

In the next cell, - Construct a view named **movie_budget** containing one row for each movie, which has (**budget**, **movie_id**, **title**), where **budget** is the numeric dollar amount extracted as a float. - To take a look at our cleaned data, write a **SELECT** query to display the **top 10 highest budget films** from **movie_budget**. When multiple films have the same budget, break ties by **movie_id** (ascending).

Hint: The query here should be quite similar to Question 3a. Make sure to break ties properly!

```
In [143]: %%sql --save query_3b result_3b <<

DROP VIEW IF EXISTS movie_budget;
CREATE VIEW movie_budget AS

SELECT
    MAX(CAST(substring(regex_replace(movie_info_sample.info,',',' ','g'),'$(\d+)') AS FLOAT))
    movie_id,
    MAX(title) AS title

FROM movie_sample

JOIN movie_info_sample ON movie_sample.id = movie_info_sample.movie_id

WHERE info_type_id = 105 AND info LIKE '$%'
```

```

GROUP BY movie_info_sample.movie_id

HAVING MAX(CAST(substring(regexp_replace(movie_info_sample.info, ',', '', 'g'), '\$(\d+)') AS FLOAT)) > 2500000000.0

ORDER BY budget DESC, movie_id ASC;

SELECT * FROM movie_budget LIMIT 10;

```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

10 rows affected.

```

In [144]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_3b = %sqlcmd snippets query_3b
grading_util.save_results("result_3b", query_3b, result_3b)
result_3b

```

```

Out[144]: +-----+-----+-----+
| budget | movie_id | title |
+-----+-----+-----+
| 300000000.0 | 2204343 | Pirates of the Caribbean: At World's End |
| 260000000.0 | 2332419 | Tangled |
| 258000000.0 | 2305993 | Spider-Man 3 |
| 250000000.0 | 1938937 | Harry Potter and the Half-Blood Prince |
| 250000000.0 | 2002374 | John Carter |
| 250000000.0 | 2204347 | Pirates of the Caribbean: On Stranger Tides |
| 250000000.0 | 2360588 | The Dark Knight Rises |
| 250000000.0 | 2387922 | The Lone Ranger |
| 237000000.0 | 1704289 | Avatar |
| 230000000.0 | 2344435 | The Amazing Spider-Man |
+-----+-----+-----+

```

```

In [145]: grader.check("q3b")

```

```

Out[145]: q3b results: All test cases passed!

```

8.4 Question 3c

We have all the parts we need to calculate the profits. Using the `movie_gross` and `movie_budget` views created above, we can now subtract the numeric columns and save the result in another column called `profit`.

In the next cell, construct a view named `movie_profit` containing one row for each movie, which has (`movie_id`, `title`, `profit`), where `profit` is the result of subtracting that movie's `budget` from `gross`. Following the view definition, write a `SELECT` query to return the **first 10 rows** of the view ordered by descending `profit`. This may take a while to execute.

```
In [148]: %%sql --save query_3c result_3c <<

DROP VIEW IF EXISTS movie_profit;

CREATE VIEW movie_profit AS

SELECT
    movie_gross.movie_id,
    movie_gross.title,
    (movie_gross.gross - movie_budget.budget) AS profit
FROM movie_gross
NATURAL JOIN movie_budget;

SELECT *
FROM movie_profit
ORDER BY profit DESC
LIMIT 10;
```

Running query in 'postgres://jovyan@127.0.0.1:5432/imdb'

10 rows affected.

```
In [149]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_3c = %sqlcmd snippets query_3c
grading_util.save_results("result_3c", query_3c, result_3c)
result_3c
```

```
Out[149]: +-----+-----+-----+
| movie_id |      title      |  profit  |
+-----+-----+-----+
| 1704289  |      Avatar     | 523507625.0 |
| 2438179  |      Titanic    | 458672302.0 |
| 2310522  |      Star Wars  | 449935665.0 |
```


1851357	E.T. the Extra-Terrestrial	424610554.0	
2346436	The Avengers	403357910.0	
2360583	The Dark Knight	349858444.0	
2400712	The Passion of the Christ	340782930.0	
2006991	Jurassic Park	338820792.0	
2172509	Olympus Has Fallen	330824682.0	
2379293	The Hunger Games	330010692.0	
+-----+-----+-----+			

```
In [150]: grader.check("q3c")
```

```
Out[150]: q3c results: All test cases passed!
```

8.5 Question 3d

We analyzed the data, but something seems odd. Upon closer look, there are many negative values for profit. For example, the movie 102 Dalmations looks to have lost around \$18M, but it was a widely successful film! What may account for this issue? Think about how we constrained our data from the start of the problem.

It may be that the currency we use is limited to US dollars, and there is also a time difference, which leads to data limitations. This affects the amount of income, while the budget is fixed.

9 Question 4: Using Cleaned Data

Now that we have cleaned our monetary records from the `info` attribute in `movie_info_sample`, let's take a closer look at the data we generated.

9.1 Question 4a: Earnings per Genre

Another `info_type` we can look at is the movie genre. Looking at the `movie_gross` values, how much does each *genre* earn on average in the US?

- Create a view with the columns `movie_id`, `title`, `gross`, `genre`, and `average_genre` where `gross` is a movie's gross US earnings, `genre` is the movie's genre, and `average_genre` is the average earnings for the corresponding genre. If a movie has multiple genres, the movie should appear in multiple rows with each genre as a row.
- Following the view definition, write a `SELECT` query to return the rows for the movie "Mr. & Mrs. Smith" ordered by genre alphabetically.

Hint: Look into [window functions](#)

```
In [151]: %%sql --save query_4a result_4a <<

DROP VIEW IF EXISTS movie_avg_genre;
CREATE VIEW movie_avg_genre AS

SELECT
movie_id,
title,
gross,
movie_info_sample.info AS genre,
AVG(movie_gross.gross) OVER (PARTITION BY movie_info_sample.info) AS average_genre

FROM movie_info_sample

NATURAL JOIN movie_gross

WHERE movie_info_sample.info_type_id = (SELECT id FROM info_type WHERE info = 'genres');

SELECT movie_id, title, gross, genre, average_genre
FROM movie_avg_genre
WHERE title = 'Mr. & Mrs. Smith'
ORDER BY genre;
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

3 rows affected.

```
In [152]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_4a = %sqlcmd snippets query_4a
grading_util.save_results("result_4a", query_4a, result_4a)
result_4a
```

```
Out[152]: +-----+-----+-----+-----+-----+
| movie_id | title | gross | genre | average_genre |
+-----+-----+-----+-----+-----+
```

2132092	Mr. & Mrs. Smith	186336103.0	Action	42123826.131625965	
2132092	Mr. & Mrs. Smith	186336103.0	Comedy	21583843.81801513	
2132092	Mr. & Mrs. Smith	186336103.0	Romance	18470817.081399772	

+-----+-----+-----+-----+-----+

```
In [153]: grader.check("q4a")
```

```
Out[153]: q4a results: All test cases passed!
```

9.2 Question 4b: Analyzing Gross Earnings

A common way to view numerical data is with a boxplot. A boxplot shows a spread of the data along with several other key attributes that allow for further data analysis.

We went through a lot of work transforming the gross earnings from strings in the `info` attribute into a numerical value. Because of our hard work, we can now further examine this data and understand its distribution. To do this, we first need to generate a [five-number summary](#) and find the average of the US gross earnings data.

- Create a view named `earnings_summary`, which consists of a one row summary of the `movie_gross` data with the min, 25th_percentile, median, 75th_percentile, max, and average.
- Following the view definition, write a `SELECT` query to display it.

Hint: Look at SQL [aggregate functions](#). You may find some useful.

```
In [154]: %%sql --save query_4b result_4b <<

DROP VIEW IF EXISTS earnings_summary;
CREATE VIEW earnings_summary AS

SELECT
MIN(gross) AS min,
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY gross) AS "25th_percentile",
PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY gross) AS "median",
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY gross) AS "75th_percentile",
MAX(gross) AS max,
AVG(gross) AS average

FROM movie_gross;
SELECT * FROM earnings_summary;
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

1 rows affected.

```
In [155]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_4b = %sqlcmd snippets query_4b
grading_util.save_results("result_4b", query_4b, result_4b)
result_4b
```

```
Out[155]: +-----+-----+-----+-----+-----+-----+
| min | 25th_percentile | median | 75th_percentile | max | average |
+-----+-----+-----+-----+-----+-----+
| 30.0 | 166623.0 | 2317091.0 | 20002717.5 | 760507625.0 | 19594424.63641884 |
+-----+-----+-----+-----+-----+-----+
```

```
In [156]: grader.check("q4b")
```

Out[156]: q4b results: All test cases passed!

9.3 Question 4c

What do you notice about the summary values generated in `earnings_summary`? We can represent the five-number summary graphically using a [box plot](#). Identify two properties about the boxplot of the data. (You do not need to explicitly create a boxplot, but think about how the summary statistics would be distributed in a boxplot.)

Hint: Think in terms of about concepts from statistics like spread, modality, skew, etc. and how they may apply here.

Skewness: This means the data is skewed to the right. The maximum value is well above the 75th percentile, further emphasizing the presence of outliers at the high end of total revenue. These outliers represent blockbusters that make more money than most other movies.

Spread: A measure of the distribution of data in the middle 50%. The difference between the maximum and the 75th percentile, and the difference between the 25th and minimum values, shows that there is a big difference in movie earnings between the top earners and the lower earners.

```
In [157]: # optional: include your plotting code here
```

10 Question 5: Joins

Joins are a powerful tool in database cleaning and analysis. They allow for the user to create useful tables and bring together information in a meaningful way.

There are many types of joins: inner, outer, left, right, etc. Let's practice these in a special scenario.

You are now working as a talent director and you need a list of all people who have been in **actor** roles and the number of movies in which they have acted.

- Create a view called `number_movies`, which has columns `id`, `name`, `number` where `id` is the actor's id, `name` is the actor's name, and `number` is the number of movies they have acted in.
- Following your view, write a `SELECT` query to display the **top 10 actors** who have been in the most films.

Note: The `cast_sample` may include actors not included in `actor_sample` table. We still want to include these actors in our result by reference to their id. The `name` field can be NULL.

```
In [158]: %%sql --save query_5 result_5 <<

DROP VIEW IF EXISTS number_movies;
CREATE VIEW number_movies AS

SELECT cast_sample.person_id AS id,
       actor_sample.name AS name,
       COUNT(cast_sample.movie_id) AS number

FROM actor_sample

RIGHT JOIN cast_sample ON cast_sample.person_id = actor_sample.id
WHERE actor_sample.gender = 'm'

GROUP BY cast_sample.person_id, actor_sample.name;

SELECT id, name, number
FROM number_movies
ORDER BY number DESC
LIMIT 10;
```

Running query in 'postgresql://jovyan@127.0.0.1:5432/imdb'

10 rows affected.

```
In [159]: # Do not delete/edit this cell!
# You must run this cell before running the autograder.
query_5 = %sqlcmd snippets query_5
grading_util.save_results("result_5", query_5, result_5)
result_5
```

```
Out[159]: +-----+-----+-----+
|      id      |      name      | number |
+-----+-----+-----+
|    95397     |    Barker, Bob  |   6853 |
|    515315    | Freeman, Morgan |   5938 |
|    677696    | Hinnant, Skip   |   4697 |
|    1573853   | Trebek, Alex    |   4690 |
|    1362169   | Sajak, Pat      |   3937 |
|    1417394   | Shaffer, Paul   |   3546 |
|    911160    | Lima, Pedro     |   2911 |
|    900749    | Letterman, David |   2895 |
|    487253    | Filipe, Guilherme |  2861 |
|    356575    | Davidson, Doug  |   2760 |
+-----+-----+-----+
```

```
In [160]: grader.check("q5")
```

```
Out[160]: q5 results: All test cases passed!
```

11 Congratulations! You have finished Project 1.

The below code prepares all the additional files needed for your submission, including: `* results.zip *`
`proj1.pdf`

Make sure to run this cell before exporting the final zip file with `grader.export()`!

```
In [161]: grading_util.prepare_submission_and_cleanup()
!otter export -e latex proj1.ipynb proj1.pdf # Export PDF
```

```
/srv/conda/envs/notebook/lib/python3.11/site-packages/nbconvert/utils/pandoc.py:51: RuntimeWarning: Your
Your version must be at least (2.14.2) but less than (4.0.0).
Refer to https://pandoc.org/installing.html.
Continuing with doubts...
check_pandoc_version()
```

```
In [162]: # Close SQL magic connection
          %sql --close postgresql://127.0.0.1:5432/imdb
```

RuntimeError: Could not close connection because it was not found amongst these: ['postgresql://jovyan@127.0.0.1:5432/imdb']
If you need help solving this issue, send us a message: <https://ploomber.io/community>

11.1 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

After you have run the cell below and generated the zip file, you can download your PDF here.

```
In [ ]: # Save your notebook first, then run this cell to export your submission.
        grader.export(pdf=False, run_tests=True, files=['proj1.pdf', 'results.zip'])
```

Running your submission against local test cases...